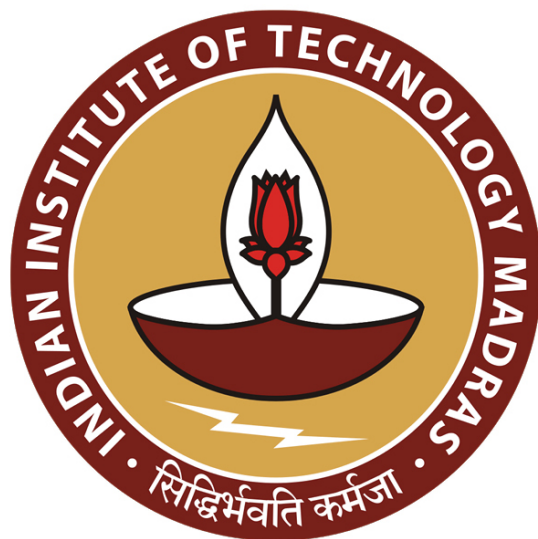
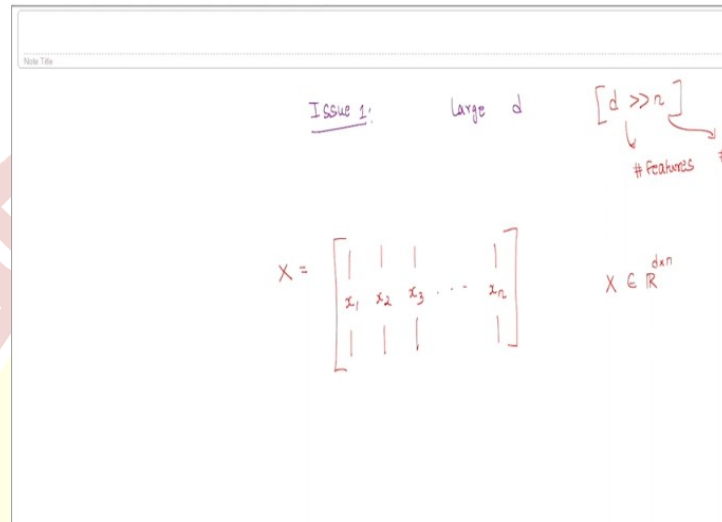




IIT Madras
ONLINE DEGREE

Machine Learning Techniques
Professor Arun Rajkumar
Department of Computer Science and Engineering
Indian Institute of Technology, Madras
Time-complexity issue with PCA

(Refer Slide Time: 00:15)



What is issue 1? It is a large d . In particular you can think of d is much much larger than n . So, d is the number of features and n is the number of data points. So, let us do some, I mean notational simplification. This will really help us, solving this issue. So, let us start by giving some, defining some matrices. The first matrix is the matrix of the data points.

Let us say we have the data set which has $x_1, x_2, x_3, \dots, x_n$. So, let us say we stack these data points as vectors, d dimensional vectors in columns next to each other. So, that is my data set itself represented as a matrix which is a $d \times n$ matrix. Now, the covariance matrix, we know

$$\text{is } \frac{1}{n} \sum_{i=1}^n x_i x_i^T.$$

(Refer Slide Time: 01:33)

The image shows a handwritten derivation of the covariance matrix C . At the top, a matrix X is defined as a $d \times n$ matrix with columns $x_1, x_2, x_3, \dots, x_n$. To the right, it is noted that $X \in \mathbb{R}^{d \times n}$. Below this, the product XX^T is shown as a $d \times d$ matrix. The first part of the product is a column vector $[x_1, \dots, x_n]^T$ (representing the transpose of X) multiplied by a column vector $[x_1, \dots, x_n]$. This is then equated to a boxed formula: $C = \frac{1}{n} XX^T$.

Now, I want to write this covariance matrix in terms of this matrix that I have defined here and that is possible, not too hard to see. So, if you look at XX^T , now, which means that means that you are taking a $d \times n$ matrix and then multiplying it with an $n \times d$ matrix which is the transpose of the data points where the data points are all rows now.

Now, this is exactly $\frac{1}{n} \sum_{i=1}^n x_i x_i^T$. So, this is perhaps you are already seen why this is the k 's, if not try to show this. So, try to show this exercise. All I am saying is that your covariance matrix which is $\frac{1}{n} \sum_{i=1}^n x_i x_i^T$, this summation can actually be expressed succinctly in matrix notation as $\frac{1}{n} \sum_{i=1}^n x_i x_i^T$ transpose. That just implies that our covariance matrix is just $\frac{1}{n} XX^T$.

So, say any d check, X is $d \times n$, X^T is $n \times d$, XX^T is $d \times d$ which is also the dimension of our covariance matrix. Of course, we are dividing it by n that is needed I mean standard definition needs you to either divide by n or $(n - 1)$ does not really matter, we will stick to n at least in this course. So, now, what does PCA do? PCA tries to find the Eigen vectors and Eigen values of the covariance matrix or find the best-fit line which happen to be Eigen vectors and Eigen values of the covariance matrix.

(Refer Slide Time: 03:24)

The slide contains handwritten text in green and red ink. At the top, there is a small red arrow pointing right and a red bracket labeled $\frac{1}{n}$. The main text in green says: "Let w_k be the eigenvector corresponding to eigenvalue of C (λ_k)". Below this, in red, is the equation $C w_k = \lambda_k w_k$ with "[by definition]" written to its right. At the bottom, in red, is the equation $\left(\frac{1}{n} \sum_{i=1}^n x_i x_i^T \right) w_k = \lambda_k w_k$.

So, let us say w_k be the Eigen vector corresponding to the k 'th largest Eigen value of C and let us call this Eigenvalue λ_k . Now, you have a matrix C and then I am saying that you take the Eigen values of this matrix, arrange them in decreasing order, λ_1 is the highest λ_2 is the second highest and so on. And w_k corresponds to the Eigenvector associated with λ_k .

Now, what is the equation that an Eigenvector satisfies and Eigenvector is a special direction for a matrix where if the matrix acts on this vector it just scales this vector by some amount. It does not change the direction. So, the direction is either scaling, it could reverse the direction. So, scaling could be negative that is still okay but it does not change the direction.

So, which means in equations, it means that $C w_k = \lambda_k w_k$. So, that is the definition of Eigen vector and Eigen value. Now, what we want to understand is, we want to find these Eigen directions which are w_1 to w_k . Now, what can we say about these w_k 's? Where do these w_k 's live? Where do these Eigen directions live?

Of course, they are all d dimensional vectors but can we say something more about where these Eigenvectors live? That is what we are attempting to find now, Now, for that what we will do is we will replace C with its definition which is $\left(\frac{1}{n} \sum_{i=1}^n x_i x_i^T \right) w_k = \lambda_k w_k$.

(Refer Slide Time: 05:50)

$$C w_k = \lambda_k w_k \quad [\text{by defini}]$$
$$\left(\frac{1}{n} \sum_{i=1}^n x_i x_i^T \right) w_k = \lambda_k w_k$$
$$w_k = \frac{\sum_{i=1}^n \left(\frac{x_i x_i^T w_k}{n \lambda_k} \right) x_i}{1}$$

w_k is a LINEAR COMBINATION

Now, what I want to do is this is basically algebra, I will retain this w_k on the left-hand side, bring the λ_k to the other side and combine this x_i and w_k together. I can bring, essentially, bringing that this w_k inside, it does not depend on i so, it can go inside. So, this whole thing becomes $w_k = \frac{1}{n \lambda_k} \sum_{i=1}^n (x_i^T w_k) x_i$.

If you are not immediately seeing why these two equations mean the same thing, pause this video, just work it out it is. I mean you should be able to convince yourself that these two things are exactly the same. It is just one-step of algebra. Now, mean if you stare at this equation for a while, something interesting becomes clear. So, this is kind of seeing.

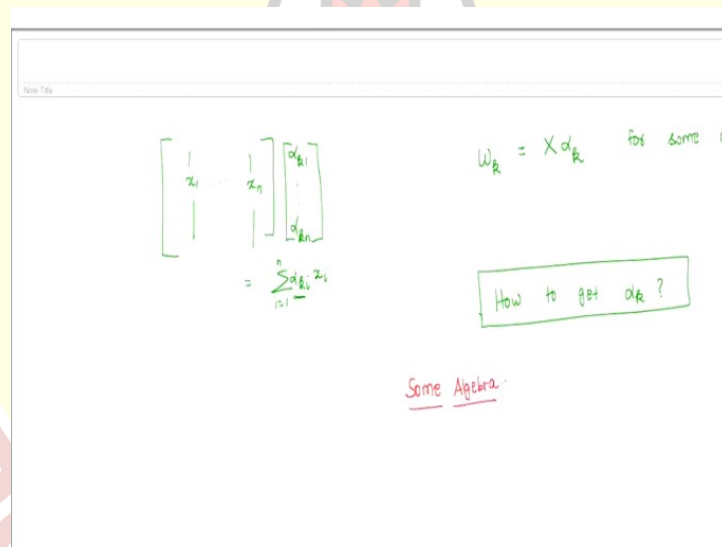
Now, let me, actually, even I can put this in λ_k inside. So, it does not really matter. So, I will tell you why I do this, yeah. So, both are exactly the same. It is a constant. I mean it does not depend on i , you can pull it inside. Now, what is this saying? This is telling me that if I want the k 'th Eigen direction, so, Eigen direction corresponding to the k 'th largest Eigenvalue.

Now, I can express that as a summation of some constant times x_i which means I take the x_i data point multiplied with some number and then add up all these scaled versions of x_i . In other words, this is essentially telling me that my w_k is a linear combination of data points. So, we are going to assume λ_k is not 0 that. So, let us do that without loss of generality in this particular k 's. So, because otherwise this is going to not, I mean, this is going to be infinity and then that would not make sense. But for $\lambda_k \neq 0$ this is true.

Now, the interesting thing is that, w_k is a linear combination of data points. Which means that somehow you need to combine your data points to get your Eigen directions. Now, for different Eigenvectors the way you combine these data points are going to be different. Now, what we care about in PCA is to get these Eigen directions which means now we can say that equivalently we can care about getting these combinations of these data points which give these Eigenvectors.

Because once I have the Eigen vectors then I know how to get a compressed representation and all that, I can project each data point on to the Eigenvectors we know that. We know what to do once we have w_k 's. But now, to get these w_k 's itself what we are saying is that it suffices to find that combination of these data points that will give me the w_k 's. So, what does that mean?

(Refer Slide Time: 09:16)



The slide contains the following handwritten content:

$$\begin{bmatrix} z_1 \\ \vdots \\ z_n \end{bmatrix} \begin{bmatrix} a_{k1} \\ \vdots \\ a_{kn} \end{bmatrix} = \sum_{i=1}^n a_{ki} z_i$$
$$w_k = X a_k \quad \text{for some } a_k$$

How to get a_k ?

Some Algebra:

$$C w_k = \lambda_k w_k \quad [\text{by defini}]$$

$$\left(\frac{1}{n} \sum_{i=1}^n x_i x_i^T \right) w_k = \lambda_k w_k$$

$$w_k = \frac{\sum_{i=1}^n \left(\frac{x_i^T w_k}{n \lambda_k} \right) x_i}{1}$$

w_k is a LINEAR COMBINATION

That means that we can say our w_k equals our matrix X , which is remember the matrix of the data point stacked in columns multiplied by some α_k for some α_k in $\mathbb{R}^n$. Now, what does this mean? This simply means that remember, our data point is like this. So, x_1 to, sorry, there matrix X is like this x_1 to x_n .

Now, I am saying there is some α_k which is in $\mathbb{R}^n$ which means that it gives some weight to each of this data point. The k says that it corresponds, these weights corresponds to the Eigenvector w_k . So, α_{k1} to α_{kn} . Now, if I multiply this, this is just $\frac{1}{n} \sum_{i=1}^n \alpha_{ki} x_i$.

Of course, we know what this α_{ki} is. So, this equation tells us that these weights are exactly $\frac{(x_i^T w_k)}{n \lambda_k}$. But then to get these weights from this equation, it appears that you already need to know w_k . So, this is $\frac{(x_i^T w_k)}{n \lambda_k}$, you need to know w_k to find w . That is a chicken and egg problem. We cannot use this equation to directly find these weights because this equation needs what we are trying to find in the first place which is w_k .

So, let us leave this equation out. We are saying here that is there a different way that we can somehow find this α_k 's. We first recognize that there are these α_k 's which exactly is this but then let us forget that it is this for the moment. But is there a different way you can somehow get these α s.

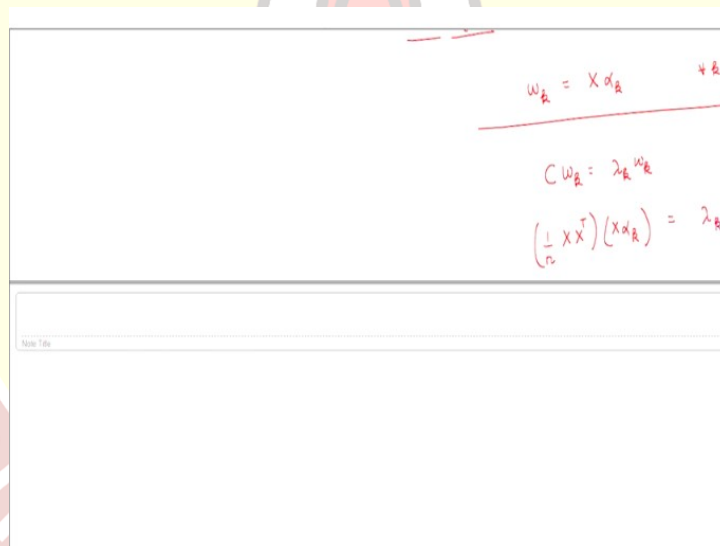
Because, if these weights, how should I weigh these data points to combine them to get my w_k that is all I need. So, I to get my w_k . So, somehow if I can get these α s then I am done.

And there is an α_k for each k . So, remember that. Because for each w_k there is a different set of weights. You have to combine the data points differently to get our w_k 's.

Now, how can we get these α_k 's? That is the how to get α_k 's, this is the question. Once we have this, once if we can somehow efficiently get α_k 's which does not require order of d^3 computation then we are in business. Because the whole point was our Eigen vector solvers are going to take order of d^3 where d is the dimension, d is the dimension or the number of features.

If we can somehow get α without spending d^3 time then that is a good thing to do. So, how can we do that is the question. So, we will do some algebra here. It is more the algebra I will do it for completion sake but then what comes out of it is more important. But I would definitely suggest you to try and follow the algebra as we do it.

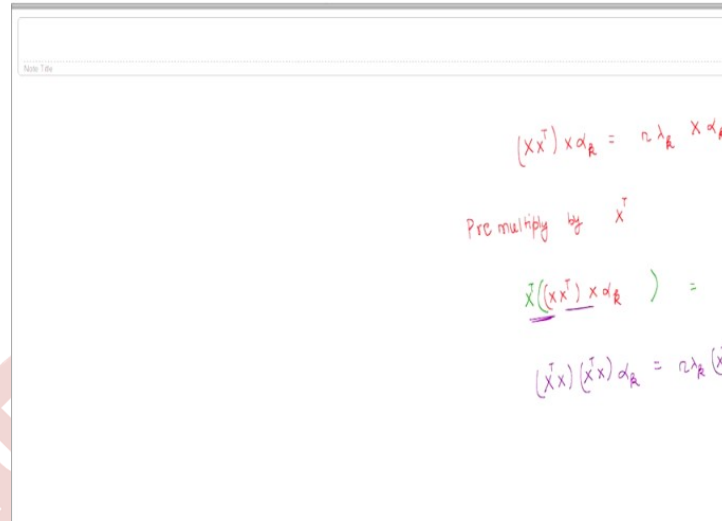
(Refer Slide Time: 12:27)


$$w_k = X \alpha_k$$

$$C w_k = \lambda_k w_k$$
$$\left(\frac{1}{n} X X^T \right) (X \alpha_k) = \lambda_k$$

So, we have $w_k = X \alpha_k$. We know that. We do not know what α is but we know that there will exist some α which is what we are trying to find. So, let this be right here. So, we know $C w_k = \lambda_k w_k$. That is by definition of the Eigenvector, w_k is an Eigenvector. It has to satisfy this. We wrote C as this, $\frac{1}{n} X X^T$. this is something that we wrote earlier. We also are saying $w_k = X \alpha_k$. That is what we observed in just a minute ago. So, this is $\lambda_k X \alpha_k$.

(Refer Slide Time: 13:16)



The image shows a whiteboard with handwritten mathematical equations. The first equation is $(XX^T)X\alpha_k = n\lambda_k X\alpha_k$. Below it, the text "Pre multiply by X^T " is written. The next line shows the result of pre-multiplying: $X^T(XX^T)X\alpha_k =$. The final line shows the simplified equation: $(X^T X)(X^T X)\alpha_k = n\lambda_k (X^T X)\alpha_k$.

Let me bring the n to the other side and write this as $XX^T(X\alpha_k) = n\lambda_k X\alpha_k$. Now, what I would do at this step is pre-multiply this equation by X^T . In other words, I am saying, I will do X^T times whatever was there equals X^T times whatever was there. And what is there was $XX^T(X\alpha_k)$ and we will see why this is useful in a minute, $X\alpha_k$.

Now, if I rearrange terms, this is I can do this, I mean I cannot, matrix multiplication is not commutative always, so, I cannot swap terms but I can change the brackets. So, it is associative. So, I can change the brackets however I wish. In other words, I can do it $(X^T X)(X^T X)$. Basically, I am combining these two guys and these two guys into α_k equals $n\lambda_k$ is a constant that can come outside. This X^T multiplies this X , $(X^T X)\alpha_k$.

(Refer Slide Time: 14:39)

Pre multiply by X^T

$$\underline{X^T(XX^T)X\alpha_R} = \underline{X^T n_R}$$

$$\underline{(X^T X)\alpha_R} = n_R \underline{(X^T X)}$$

Call $X^T X := K$

$$K\alpha_R = n_R K$$

Now, this is a matrix. Now, remember X was in $\mathbb{R}^d \times n$. So, $(X^T X)$ is in, this is $X^T n \times d$, x is $d \times n$, so, this is $n \times n$. So, that is just to remember. Now, call $X^T X$ let us just give it a name, let us call it K . Then this equation is $K^2 \alpha_k = (n \lambda_k) \alpha_k$. So, we want the α_k somehow and we are saying whichever α_k that is that you need to use to combine these data points to get w_k should satisfy the equation $K^2 \alpha_k = (n \lambda_k) \alpha_k$.

(Refer Slide Time: 15:37)

So we can find α_R that

$$K\alpha_R = (n\lambda_R)\alpha_R$$

$$K u = (n\lambda_R) u$$

$$K(2u) = (n\lambda_R)(2u)$$

In other words, if we can find α_k that satisfies, there is a k on both sides, so, I am saying, if we can find an α_k that satisfies $K\alpha_k = (n\lambda_k)\alpha_k$ then we can multiply by K on both sides and it

would have satisfied the other equation as well. So, which means that all we need to find is an α_k that satisfies $K\alpha_k = (n\lambda_k)\alpha_k$. If we can do that then we are kind of done.

Now, this looks like an Eigen equation. So, this looks like an Eigen equation. In fact, this is an Eigen equation. So, basically, we are saying that, if you take α_k whatever this α_k is, if I apply this K matrix to this α_k , it has to scale this α_k by $n\lambda_k$. So, if I can find such an α_k then I am done. So, this is an Eigen equation. Now, let us say, I give you an α_k . So, I give you something and then claim that, that satisfies this equation.

So, let us say I give you some vector u and then it satisfies this equation that $Ku = (n\lambda_k)u$, let us say. Now, is this u unique? No. So, now, what I can do is I can do $K2u$ will satisfy $(n\lambda_k)2u$. Now, what does this tell us that every, I mean, I can scale this u by any number and then it will still satisfy this equation.

So, which means that we need a specific way of combining these data points. So, specific alpha that will give us a w_k . We know that the length of w_k is 1. So, we were looking for directions with length 1. Now, which means that there should be something that we can say about the length of α_k also or in general α_k also. So, which means that you cannot arbitrarily scale this u that satisfies this equation and claim that all of these are α_k 's. So, now, what is that? What does that tell us is what we are trying to find out?

(Refer Slide Time: 18:14)

So we can find α_k that

$$K \alpha_k = (n \lambda_k) \alpha_k$$

We know $w_k = X \alpha_k$

$$w_k^T w_k = (X \alpha_k)^T (X \alpha_k) =$$

$$1 = \alpha_k^T K \alpha_k$$

$(X^T X) (X \alpha_k) = n \lambda_k (X \alpha_k)$

Call $X^T X := K$

$$K \alpha_k = n \lambda_k \alpha_k$$

So we can find α_k that

$$K \alpha_k = (n \lambda_k) \alpha_k$$

Let us try to find out how the length of w_k gives us an indication of what is the length that we should look for, for α_k . What do we know? So, we know $w_k = X \alpha_k$. We saw that. So, we know that w_k is some combination of this which means $w_k^T w_k$ which is the length is $(X \alpha_k)^T (X \alpha_k)$. Now, this is $\alpha_k^T (X^T X) \alpha_k$. We know that we are looking for w_k 's with the length 1 which means the left-hand side is 1. On the right-hand side, this $(X^T X)$ shows up which is what we are calling as K . Which means this is α_k .

So, now, we are saying that we need an α_k that satisfies this equation but it can not be any α_k , such an α_k should also satisfy $\alpha_k^T K \alpha_k = 1$. If you find an α_k that satisfies this equation and if that α_k satisfies $\alpha_k^T K \alpha_k = 1$ then that is the α_k that corresponds to w_k which has length 1.

So, all this is algebra. All that we are saying is that we wanted w_k , we are saying we can equivalently solve for α_k . And solving for α_k looks like solving for an Eigen equation in K . But then the Eigenvectors length is unspecified. So, we need to normalize the length and then the fact that w_k 's of length 1 implies that $\alpha_k^T K \alpha_k = 1$. So, we should look for α_k 's Eigen equations but then you should also have this property. So, this is first point.

(Refer Slide Time: 20:42)

Handwritten notes on a whiteboard background with a large circular watermark reading "INDIAN INSTITUTE OF TECHNOLOGY MADRAS".

Top note (boxed): $1 = \alpha_k^T K \alpha_k$

Middle note: LINEAR ALGEBRA FACT : The non-zero eigenvectors are exactly the same!

Bottom note: $\sum \alpha_k^T K \alpha_k = \sum \lambda_k \alpha_k^T \alpha_k$

Bottom note (boxed): $K \alpha_k = (\lambda_k) \alpha_k$

Bottom note: $\alpha_k^T K \alpha_k = \lambda_k \alpha_k^T \alpha_k$

So, which means, so, now, we need one more important theorem from linear algebra which will actually be very very useful in understanding, in solving, in completing this problem. And that fact, I mean this is a linear algebra fact or actually a theorem but I will state it as a fact for now. And we will see why this fact is useful.

So, essentially before I state this fact, so, let me say why we need this fact. Essentially, we wanted the Eigenvectors of XX^T but now we are saying we can solve the Eigen equation of K , where K is just X^TX . Now, somehow this also involves λ_k . We need to know λ_k which is the Eigen value of XX^T . But then we are only solving the Eigen equation for X^TX .

So, now, is there a relation between the Eigen values of XX^T and X^TX ? So, because if we solve the Eigen equation for K that will give you a set of Eigen vectors and Eigen values, how are these Eigen values related to the Eigen values that you would have gotten had you solved the Eigen equation for XX^T . That is the question we are asking.

And the answer is this linear algebra fact. The non-zero Eigenvalues of XX^T and X^TX are exactly the same. So, now, to make it precise, so, XX^T is a matrix in $d \times d$, X^TX is a matrix in $n \times n$. But because both of these come from the underlying x which is $d \times n$ their Eigen values are related.

In fact, if you are aware of the singular value decomposition of matrices then you can simply use that to prove this in two steps. I would not do that. Feel free to try this. But what I am saying is that the non-zero Eigenvalues, there might be 0 Eigen values because this is $d \times d$, this is $n \times n$, so, the maximum number of non-zero Eigen values of these matrices will be the minimum of d and n . That is again a linear algebra fact.

So, there is going to be a lot of zero Eigen values if d is large. But which there would not be corresponding Eigen values in n . So, if n is smaller than d then and if your matrix has full length, so, then your number of non-zero Eigen values will be n and they will match with the top n Eigen values of XX^T which is a $d \times d$ matrix. So, what does this essentially tell us? So, what is all of this telling us now? It is telling us the following. And this is the most important thing.

(Refer Slide Time: 23:44)

Handwritten notes on a whiteboard:

$$C = \frac{1}{n} XX^T$$
$$\text{Eigenvectors} = \{w_1, \dots\}$$
$$\text{Eigenvalues} = \{\lambda_1 \geq \lambda_2 \geq \dots\}$$
$$XX^T = nC$$
$$\text{Eigenvalues} = \{n\lambda_1 \geq n\lambda_2 \geq \dots\}$$

We will now try to put everything together and see all of these together. So, we initially had C which was $\frac{1}{n}XX^T$ whose Eigen vectors and Eigen values we wanted. So, let us say the Eigenvectors of C where w_1 to w_n , some w_i which we want. So, now, for all k in from 1 to n we know w_k is 1, the length is 1.

Now, the Eigen values corresponding to this are λ_1 , in fact, they are in decreasing order, $\lambda_1 > \lambda_2 > \dots > \lambda_n$. So, this is what we wanted to get. Now, X , let us look at XX^T , XX^T is just nC . Now, what will be the Eigen vectors of length 1 for XX^T .

They can they will exactly be again w_1 to w_n . It is just X , the matrix is just scaled. So, the Eigen values will scale accordingly but the Eigenvectors of length 1 will stay the same. The Eigen values will $n\lambda_1 > n\lambda_2 > \dots > n\lambda_n$. So, we need this w_1 to w_n somehow.

(Refer Slide Time: 25:17)

$$\frac{X^T X}{K}$$

Eigenvectors = $\{\beta_i\}$

Eigenvalues = $\{n\lambda_i\}$

$$K \beta_k = (n\lambda_k) \beta_k$$

So we can find α_k that

$$K \alpha_k = (n\lambda_k) \alpha_k$$

We know $w_k = X \alpha_k$

$$w_k^T w_k = (X \alpha_k)^T (X \alpha_k) = \alpha_k^T X^T X \alpha_k$$

Now, what are we saying? We are saying we need to solve an Eigen equation of $X^T X$. This is the matrix which we care about. Now, let us say the Eigenvectors of $X^T X$ happen to be some vectors β_1 to β_l , some Eigenvectors. Now, what we know? We know that all the Eigenvectors length is 1. All of these guys are of length 1 and let us say, now, what are the corresponding Eigen values.

Now, the fact, linear algebra fact that I stated before says that XX^T and $X^T X$ have exactly the same non-zero Eigenvalues. Which means the Eigen values of $X^T X$ will also be $n\lambda_1 > n\lambda_2 > \dots > n\lambda_l$. So, this we have. So, now, what does this mean? This means that this is K.

So, which means that $K\beta_k = (n\lambda_k)\beta_k$. So, we have found an Eigen vector β_k which satisfies $K\beta_k = (n\lambda_k)\beta_k$. Now, question is what we wanted. We wanted some α_k 's which satisfies $K\alpha_k = (n\lambda_k)\alpha_k$.

Now, here is something which satisfies, we found a β_k which satisfies $K\beta_k = (n\lambda_k)\beta_k$. But, is $\beta_k = \alpha_k$? Can we solve the Eigen equation like this and say is β_k same as α_k ? Well we saw that there is a length constraint also on this α_k 's. It is not just the fact that it has to satisfy this equation, it also has to satisfy $\alpha_k^T K \alpha_k = 1$.

(Refer Slide Time: 27:30)

$$\beta_k^T \times \beta_k = \beta_k^T (n\lambda_k \beta_k)$$

$$\text{Set } \alpha_k := \frac{\beta_k}{\sqrt{n\lambda_k}} \quad \text{Now}$$

So, which means, now, let us check with β_k . So, now, what is $\beta_k^T K \beta_k$, what is this value? If β_k was α_k then this has to be 1. But what is this? By the virtue of the fact that β_k is an Eigen vector of K , so, $K\beta_k$ is $(n\lambda_k)\beta_k$. So, this is $\beta_k^T (n\lambda_k)\beta_k$.

But we know β_k is of length 1. So, this is just $n\lambda_k$. So,

$$\beta_k^T K \beta_k$$

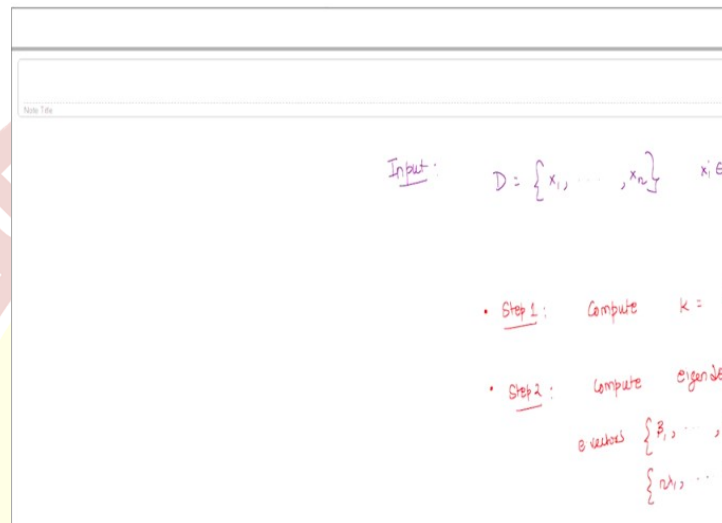
is actually $n\lambda_k$. But then we wanted, if we said β_k is α_k then it is not going to work because then no longer $\alpha_k^T K \alpha_k = 1$. So, there is a scaling of $n\lambda_k$ that is happening.

So, which means we can set α_k as $\frac{\beta_k}{\sqrt{n\lambda_k}}$. If you do this, now, if once I set this for all k , now,

$K\alpha_k = (n\lambda_k)\alpha_k$ and $\alpha_k^T K \alpha_k = 1$. Why? Because $\alpha_k^T K \alpha_k$ is simply $\frac{\beta_k^T K \beta_k}{n\lambda_k}$. But we know the numerator is $n\lambda_k$ that is what we argued here, divided by $n\lambda_k$. This is just 1.

So, now, this is kind of telling us, how to convert your Eigen solutions for this matrix k into the α_k that we really needed. We will talk about why this is a useful thing to do. So, we are kind of going in circles and trying to find α_k but then I will tell you why this is a good thing to do in a minute.

(Refer Slide Time: 29:40)



So, now, here is what is the algorithm that we are proposing. So, our input is just the data set d which is $\{x_1, \dots, x_n\}$, where all x_i 's are in $\mathbb{R}^d$ and the assumption is that d is much larger than n . So, this is the setup that we are in if we have too many features. That is where the time complexity is a problem. Now, what are we saying?

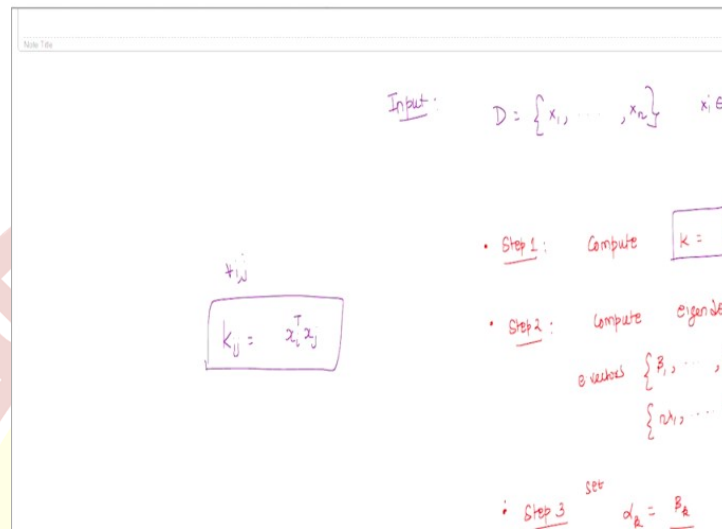
We are saying that step 1, earlier, we would have computed the covariance matrix and then we would have computed the Eigenvectors and Eigenvalues. We no longer want to do that because it is expensive. So, what is the step 1? Step 1 compute $K = X^T X$. So, K is in $\mathbb{R}^n \times n$. Step 2. Compute Eigen decomposition of K .

Now, this is still an Eigen decomposition. Well we wanted to avoid a costly Eigen decomposition but then we are saying in step 2, we are doing an Eigen decomposition of K . But note that K is an $n \times n$ matrix. Which means the Eigen decomposition of K is only going to take order of n^3 .

And if d is much much larger than n , essentially, what you are saying is that instead of doing a d^3 Eigen decomposition you can do an $n^3 d$ Eigen decomposition. So, which might be much

cheaper, in general. So, we do Eigen decomposition of K and we get Eigen vectors β_1 to β_n with corresponding Eigen values. These are Eigen vectors, $n\lambda_1 \dots n\lambda_n$.

(Refer Slide Time: 31:44)



Once you have done this, so, this is an order of n^3 computation. So, once this is done then we are almost done. So, step 3. We know that these β_k 's are not exactly α_k 's but then you have the Eigen values also with. So, what you can do is set $\alpha_k = \frac{\beta_k}{\sqrt{n\lambda_k}}$ for all k equals 1 to n . That is it.

So, once you have this then you can get back your w 's if you want. So, $w_k = X\alpha_k$. So, essentially, what we have done is we have gone in a different row vector to find our w_k 's. Instead of directly solving the Eigen decomposition, we are solving a different matrix, a related matrix, the Eigen decomposition of which is cheaper.

And then we are getting the Eigenvectors and then converting that into the weights that you need to combine the data points to get the Eigen directions of the covariance matrix. Now, this is pretty much solving problem number 1, issue number 1. Because we are not working with a huge $d \times d$ matrix but then we are working with smaller $n \times n$ matrix.

Of course, this is helpful only if d is much much larger than n . If n is larger than d , you might as do the covariance matrix decomposition. But then we are in that setup where we are assuming that d is much much larger than n and then we are trying to do this. So, this solves issue 1, the time complexity issue.

Now, note that you cannot, I mean you cannot get away with the Eigen decomposition completely. So, you have to do it. But then because there are only two important parameters in this matrix, one is d , one is n . We are just trying to make it as simpler as possible by picking the smaller one of, smaller of these. So, that is all issue 1.

Now, we want to talk about issue 2 in the next video. But before going there, I would want to make one observation which I will again make in the next video but then I want to hint it at this point and then we will come back and connect it to the next video. Now, what are we saying here, we are given this data set, the first step was to compute $X^T X$. So, what is k_{ij} in that case?

k_{ij} is $x_i^T x_j$. You can verify that this is what k_{ij} would be, so, for all i, j . In some sense, this is capturing the similarity between x_i and x_j , in the dot product sense. More importantly, this also tells us that to solve the PCA problem, you only need these pairwise dot products between the data points. If you have that you have all the information that you need to compute the PCA things.

So, whatever you want in PCA. We will make this more precise next time and then we will see that this important observation of the fact that you only need this kind of a similarity between these data points and not necessarily the data points themselves always plays a very important role in trying to use the solution of issue 1 to actually solve for issue 2 which is a totally different question. It says that what if the data points are not linearly related. It needs some creative thought to take this solution and then solve issue 2 and that is, in fact, one of the very important ideas in classical machine learning and we will talk about that in the next video.

For now, I would want to summarize saying that all we have seen today is this set of videos is to see how we have identified some issues with PCA and then how you can solve the issue of time complexity by converting your original problem into a simpler problem in a computational sense and solving the simpler problem will help you solve the original problem. We will talk more about issue 2 in the next video. Thank you.